

班 级 2218021

学 号 22009200828

西安电子科技大学

本科毕业设计论文



题 目 BAS 攻击环境模拟

学 院 网络与信息安全学院

专 业 网络工程

学生姓名 郑佳缘

导师姓名 胡建伟

摘 要

随着高校毕业论文和学术论文写作需求的不断增加，论文排版的规范性和效率逐渐成为学生和科研人员关注的重要问题。传统文字处理工具虽然操作简单，但在复杂公式、图表编号、参考文献管理和模板适配等方面存在一定不足；LaTeX 具有较强的学术排版能力，但其语法规则、编译环境和错误日志对初学者而言具有较高门槛。针对上述问题，本文设计并实现了一套基于 JavaWeb 和 LaTeX 的学术论文智能排版系统，旨在为用户提供在线化、模板化和智能化的论文写作与排版支持。

系统采用前后端分离架构，前端基于 Vue 3 和 Vite 构建用户交互界面，后端基于 Spring Boot、MyBatis 和 MySQL 实现业务逻辑处理与数据持久化。系统围绕论文写作流程，主要实现了用户注册登录、邮箱验证码、个人资料管理、论文项目管理、LaTeX 在线编辑、模板管理、论文编译、PDF 预览、文件导出、AI 对话辅助和编译错误分析等功能。用户可以在浏览器中创建论文项目，选择或导入 LaTeX 模板，在线编辑论文源码，并通过多种编译引擎生成 PDF 文档。

在系统实现过程中，针对 LaTeX 模板依赖文件较多、编译错误不易理解等问题，系统支持 zip 模板包导入，并在编译过程中尽量保留模板目录结构，减少因 .cls、.sty、.bib 等依赖文件缺失导致的编译失败。同时，系统引入 AI 辅助功能，在用户遇到 LaTeX 语法问题或编译失败时，能够根据错误日志给出原因分析和修改建议，从而降低 LaTeX 使用难度。

测试结果表明，该系统能够较好地完成论文项目管理、模板载入、在线编辑、论文编译、PDF 预览和 AI 错误分析等任务，基本满足学术论文在线排版的使用需求，具有一定的实用价值和应用前景。

关键词：关键词：自动化渗透测试 入侵与攻击模拟（BAS） 微内核架构
有限状态机（FSM） 依赖倒置 断点续传

ABSTRACT

With the increasing demand for undergraduate theses and academic paper writing in universities, the standardization and efficiency of paper formatting have become important concerns for students and researchers. Although traditional word processing tools are easy to use, they have certain limitations in handling complex formulas, figure and table numbering, reference management, and template adaptation. LaTeX has strong academic typesetting capabilities, but its syntax rules, compilation environment, and error logs present a relatively high barrier for beginners. To address these problems, this thesis designs and implements an intelligent academic paper typesetting system based on JavaWeb and LaTeX, aiming to provide users with online, template-based, and intelligent support for academic writing and typesetting.

The system adopts a front-end and back-end separated architecture. The front end is built with Vue 3 and Vite to construct the user interaction interface, while the back end is developed using Spring Boot, MyBatis, and MySQL to implement business logic processing and data persistence. Centered on the academic writing process, the system mainly implements functions such as user registration and login, email verification codes, personal profile management, paper project management, online LaTeX editing, template management, paper compilation, PDF preview, file export, AI-assisted dialogue, and compilation error analysis. Users can create paper projects in the browser, select or import LaTeX templates, edit LaTeX source code online, and generate PDF documents through multiple compilation engines.

During system implementation, considering the large number of dependency files required by LaTeX templates and the difficulty of understanding compilation errors, the system supports importing ZIP template packages and preserves the template directory structure as much as possible during compilation, thereby reducing compilation failures caused by missing dependency files such as `.cls`, `.sty`, and `.bib` files. Meanwhile, the system introduces AI-assisted functions. When users encounter LaTeX syntax problems or compilation failures, the system can analyze error logs and provide possible causes and modification suggestions, thus lowering the difficulty of using LaTeX.

The test results show that the system can effectively complete tasks such as paper project management, template loading, online editing, paper compilation, PDF preview, and AI-based error analysis. It basically meets the needs of online academic paper type-setting and has certain practical value and application prospects.

Keywords: Automated Penetration Testing Breach and Attack Simulation (BAS)
Microkernel Architecture Finite State Machine (FSM) Dependency
Inversion Breakpoint Continuation

目 录

第一章 绪论

1.1 研究背景与意义

在当今全球数字化转型加速演进的背景下，企业 IT 基础设施正经历着从传统的本地数据中心向混合云、多云及边缘计算架构的剧烈变革。这种变革在提升业务灵活性与扩展性的同时，也使得网络空间的攻击面（Attack Surface）呈现出爆炸式增长与动态演化的特征。根据全球知名网络安全研究机构 Gartner 在 2024 年发布的《安全与风险管理趋势报告》，现代企业面临的威胁已不再局限于单点的漏洞利用，而是具有长周期、高隐蔽、多阶段特征的高级持续性威胁（Advanced Persistent Threats, APT）^{gartner2024ctem}。

传统的防御体系——即以防火墙（FW）、入侵检测系统（IDS）及终端防护（EPP）为核心的静态防御架构，在面对高度组织化的对抗行为时日益显得捉襟见肘。与此同时，传统的安全评估手段，如一年一度的合规性渗透测试或自动化的漏洞扫描（Vulnerability Scanning），也暴露出了显著的时效性瓶颈。这类手段本质上是“时间点（Point-in-time）”的静态检查，无法验证安全防御体系在持续变化的攻击链路面前的真实抗性。

为此，安全验证（Validation）领域正经历着从“漏洞导向”向“验证导向”的范式转移。入侵与攻击模拟（Breach and Attack Simulation, BAS）作为一种能够持续、自动化地模拟真实黑客攻击战术的技术，已成为持续威胁暴露面管理（Continuous Threat Exposure Management, CTEM）框架中的核心环节^{crowdstrike2024bas}。CTEM 强调通过循环往复的范围界定、发现、排序、验证与动员过程，实现对风险的主动治理。

然而，在当前的工程实践中，自动化渗透测试与 BAS 系统的落地面临着严重的底层调度难题。现有的开源框架或企业自研脚本，往往采用过程化的、高度耦合的线性逻辑。这种架构在处理简单的单点攻击时表现尚可，但在推演包含“探测-提权-规避-窃密-横向”的长杀伤链（Kill Chain）时，由于控制流与物理 I/O（如网络回显、文件操作）缺乏隔离，极易出现由于目标环境不稳定性导致的逻辑死锁、状态迷失或脏数据污染。研究并设计一种具备高度工程鲁棒性、逻辑与执行严格解耦的自动化渗透调度引擎，不仅能够填补 BAS 领域在复杂场景下的调度空

白，更对于构建自动化、持续化的安全运营体系具有深远的工程价值与学术意义。

1.2 国内外研究现状

1.2.1 自动化攻击模拟基础设施的研究现状

在底层执行引擎方面，目前国际上最具代表性的开源框架是 MITRE 公司开发的 Caldera 平台。Caldera 基于 MITRE ATTCK 知识库，通过 Agent 与 C2 服务器的异步通信，实现了战术动作的自动化下发^{mitre2024caldera}。尽管 Caldera 提供了一套基于规则的规划器 (Planner)，但在真实的红队对抗模拟中，这种基于规则的调度方式过于生硬，难以处理复杂的上下文逻辑依赖 (Contextual Dependency)。除了 Caldera，Red Team Automation (RTA) 和 Atomic Red Team (ART) 等工具也提供了丰富的原子化攻击载荷。然而，这些工具大多侧重于“执行”本身，而非“调度”。它们缺乏一个能够感知渗透进度的“大脑”，导致在执行跨主机的横向移动或多步权限提升时，仍需大量的人工干预或编写极难维护的复杂脚本。

1.2.2 自动化渗透决策算法的研究现状

学术界近年来尝试引入更高级的决策模型来提升渗透测试的智能化水平。主要的研究路径包括：

(1) 基于强化学习 (RL) 的路径搜索：研究者尝试将渗透测试建模为马尔可夫决策过程 (MDP)，利用近端策略优化 (PPO) 或深度 Q 网络 (DQN) 在虚拟网络拓扑中学习最优攻击路径^{wang2024automated}。

(2) 基于大语言模型 (LLM) 的动作生成：2023 年至 2024 年，多篇顶刊论文探讨了利用 LLM 解析终端回显并自动生成后续 Payload 的可能性^{zhang2024towards}。

(3) 基于形式化方法的流程控制：一部分学者主张利用 Petri 网或有向无环图 (DAG) 对攻击行为进行建模，以保证逻辑的一致性。

1.2.3 现有研究的瓶颈与痛点

尽管上述研究在“如何选择下一个攻击动作”上取得了长足进步，但它们普遍忽略了“如何稳健地执行这个动作”的底层架构问题。现有的自动化系统在工程实现上普遍存在三个瓶颈：

(1) 逻辑与环境的强耦合：攻击脚本中充斥着大量的 API 调用、文件读写与正则匹配，任何一环的阻塞都会导致整个控制流的崩溃。

(2) 并发冲突与状态迷失：在多目标环境下，由于缺乏状态机的严格约束，调度引擎极易在并发执行中混淆不同主机的渗透进度。

(3) 容错与自愈能力缺失：传统系统缺乏原子化的状态落盘机制，一旦主进程因网络异常中断，所有的攻击上下文信息将全部丢失，无法实现断点续传。

1.3 本文的主要研究内容与贡献

针对上述背景与技术痛点，本文提出并实现了一种基于微内核有限状态机 (FSM) 的自动化渗透测试调度引擎。本文不侧重于单个漏洞的挖掘技术，而是专注于自动化渗透流程的架构控制与工程稳健性。本文的核心研究内容与创新点如下：

提出并设计了四层解耦的微内核调度架构。本文借鉴了现代操作系统的微内核设计思想，将系统切分为总控调度层、状态路由引擎层、业务算子层与运行时环境隔离层。通过将易变的渗透逻辑（算子）与稳定的核心引擎隔离，提升了系统的可维护性与扩展性。

构建了基于外置状态转移矩阵的 FSM 推演模型。引擎通过查阅静态定义的转移矩阵进行状态跃迁，彻底摒弃了复杂的条件分支语句。将所有的执行结果归约化为语义事件，实现了控制权的上收与逻辑的原子化。

实现了彻底的 I/O 隔离与副作用收口。所有的物理操作（网络请求、本地文件操作）均下沉至 Runtime 层，并引入了目标专属组隔离机制，成功解决了分布式环境下的“核裂变”感染问题，确保了攻击行为的精准性。

设计了基于状态快照的事务性持久化机制。系统实现了“每步一存”的快照策略，确保在复杂的杀伤链演进过程中，无论何时发生意外中断，系统重启后均能从上一个稳定状态完美恢复，具备了工业级的可靠性。

1.4 本文的组织结构

本文共分为五章，具体安排如下：

第一章绪论。论述研究背景、意义及国内外研究现状，明确本文的核心贡献。

第二章核心理论与相关技术。介绍 BAS 技术模型、FSM 理论、Caldera API 机制以及软件工程中的 IoC 与微内核理念。

第三章极简微内核状态机架构设计。深入剖析系统的总体架构设计思想、分层模型以及状态转移矩阵的数学定义。

第四章系统核心模块的工程实现。详细论述 Runtime 隔离封装、算子标准化构造、8 大核心 Ability 的底层原理剖析以及引擎推演逻辑。

第五章总结与展望。总结研究成果，指出系统不足并探讨未来引入 DAG 与 AI 决策的演进方向。

第二章 核心理论与相关技术

本章主要阐述支撑本文自动化渗透调度引擎设计的核心基础理论与底层技术机制。首先介绍网络安全评估从传统漏洞扫描向自动化入侵与攻击模拟 (BAS) 的演进模型；其次深入剖析作为本系统底层物理执行基座的 MITRE Caldera 框架及其异步 API 控制原语；随后，引入离散数学与计算机科学中的有限状态机 (FSM) 理论，探讨其在控制流重构中的数学优势；最后，结合现代软件工程领域的微内核架构 (Microkernel Architecture) 与依赖倒置原则 (DIP)，为后续章节的解耦系统设计奠定坚实的理论基石。

2.1 自动化渗透与入侵攻击模拟 (BAS) 模型演进

2.1.1 从原子化漏洞评估到多步杀伤链推演

长久以来，企业网络安全的有效性验证主要依赖于漏洞评估 (Vulnerability Assessment, VA) 与传统的人工渗透测试 (Penetration Testing, PT)。漏洞评估通常基于已知的通用漏洞披露 (CVE) 指纹库，对目标系统进行静态扫描。然而，这种原子化 (Atomic) 的检查方式存在显著的局限性：它只能证明“系统中存在某个弱点”，却无法证明“攻击者能够利用该弱点在复杂的网络拓扑中达成最终的破坏目标”。随着洛克希德·马丁 (Lockheed Martin) 公司提出“网络杀伤链 (Cyber Kill Chain)”模型，以及 MITRE 公司主导的 ATTCK (Adversarial Tactics, Techniques, and Common Knowledge) 框架的全面普及，安全防御的视角正式从“漏洞导向”转向了“战术与技术导向”^{strom2018mitre}。现代高级持续性威胁 (APT) 往往不依赖于单一的高危零日漏洞 (0-day)，而是通过一系列低危、合法的系统管理机制 (如 WMI、WinRM、凭据转储) 进行组合利用。这就要求自动化安全验证平台必须具备“多步杀伤链推演”的能力。

2.1.2 BAS 系统的架构特征与调度挑战

在此背景下，入侵与攻击模拟 (Breach and Attack Simulation, BAS) 技术应运而生。BAS 系统的核心理念是在真实生产环境或高度仿真的数字孪生环境中，部署受控的轻量级代理 (Agents)，并由控制端下发剧本，自动化地重放已知威胁主体的战术序列 (如：初始访问 → 执行 → 提权 → 防御规避 → 凭证窃取 →

^{alhaj2023cyber} *BASStateDependencyEnvironmentalUncertaintyIPContextControlPlane*

2.2 MITRE Caldera 对抗模拟框架与控制原语

本系统在底层采用 MITRE 公司开源的 Caldera 框架作为物理对抗模拟的执行基座。Caldera 是一个高度模块化、面向攻击者行为仿真的命令与控制（Command and Control, C2）平台^{mitre2024caldera}。了解其底层运行机制，是理解本文 API 隔离与接管逻辑的前提。

2.2.1 Caldera 的实体架构与执行模型

Caldera 严格遵循 Client-Server 异步通信模型，其业务逻辑由以下几个核心实体（Entities）驱动：

(1)Agent（代理）：运行在目标受控主机上的信标程序（Beacon），如 54ndc47。Agent 负责定期向 C2 服务器发送心跳（Heartbeat），并拉取待执行的载荷。

(2)Ability（能力）：对应 ATTCK 框架中的具体技术（Technique）。在 Caldera 中，它通常被定义为一段 YML 格式的配置文件，包含了针对特定操作系统（如 Windows）的具体 Shell、PowerShell 命令或可执行二进制文件。

(3)Adversary（对手剧本）：按照特定战术逻辑组合在一起的 Ability 集合。

(4)Planner（规划器）：Caldera 原生的决策大脑，负责决定在给定时刻应该向哪个 Agent 下发哪个 Ability。

2.2.2 原生 Planner 启发式算法的局限性

Caldera 原生的 Planner 主要基于图论与启发式规则（Heuristic Rules）进行任务调度。虽然这种方式在处理简单的无状态攻击时具备灵活性，但在需要严格时序控制的自动化横向移动场景中，却表现出严重的局限性。启发式算法本质上是非确定性的（Non-deterministic），它难以在外部控制端进行精准的、基于细粒度上下文流转的逻辑干预。一旦某个战术动作由于目标环境的变异而产生预期之外的回显，原生 Planner 极易陷入无限重试或逻辑死锁。

2.2.3 RESTful API 机制与控制权劫持

为了克服原生规划器的不可控性，本文彻底摒弃了 Caldera 的内部 Planner，转而仅将其作为一个无状态的“指令下发执行器（Executor）”。Caldera 提供了完善的 RESTful V2 API，控制端通过向 /api/v2/operations 发送 POST 请求即可触发攻击任务。必须指出的是，该 API 的通信机制是极度异步的：C2 服务器返回 HTTP 200 状态码，仅仅意味着任务已成功加入内部队列，并不代表受控端 Agent 已经执行完毕。因此，控制端必须通过轮询（Polling）/api/v2/operations/id/links 及对应的 /result 端点，才能异步提取 Base64 编码的底层命令回显。这种底层异步通信机制

为上层同步状态机的封装提出了挑战，但也为系统实现任务隔离与容灾重试提供了物理切面。

2.3 有限状态机 (FSM) 理论与控制流重构

为了从根本上解决传统渗透脚本中由深度嵌套的 `if-else` 与 `while` 语句导致的“面条式代码 (Spaghetti Code)”问题，本文在调度引擎的核心设计中引入了离散数学与计算机自动机理论中的有限状态机 (Finite State Machine, FSM) 模型 [hopcroft2006introduction](#)。

2.3.1 FSM 的数学定义与系统映射

有限状态机是一个严谨的数学计算模型，用于描述系统在有限个离散状态之间响应外部事件而发生的确定性转移过程。在形式化定义中，一个标准的确定性有限状态自动机 (DFA) 可由五元组 $\Sigma = (S, \Sigma, \delta, s_0, F)$ 表示：

S ：系统所有可能状态的有限集合（在本文中对应探测、提权、规避、窃密、横向等攻击阶段）。

Σ ：触发状态流转的输入事件集合（在本文中对应业务算子返回的 `success`、`error` 等语义结果）。

δ ：状态转移函数，映射 $S \times \Sigma \rightarrow S$ （在本文中对应引擎中的状态转移矩阵 Transition Matrix）。

s_0 ：系统的初始状态（探测阶段）。

F ：系统的终止状态集合（渗透成功或异常熔断）。

在复杂的自动化调度中引入 FSM 理论，其最大的学术与工程价值在于剥离了状态决策与状态执行。当前系统的下一步走向仅由“当前状态”与“触发事件”共同决定，外界环境的任何网络波动或异常崩溃，都可以被规约到明确的状态节点上，从而实现严格的数学意义上的可预测性与极高的系统鲁棒性。

2.3.2 降低环路复杂度与状态分离的优势

在传统的命令式编程中，程序的控制流（决定下一步去哪）与业务逻辑（怎么执行攻击）是高度纠缠的。随着渗透深度的增加，程序的圈复杂度 (Cyclomatic Complexity) 急剧上升。引入 FSM 理论的最大工程价值在于，它实现了状态决策与状态执行的绝对物理剥离。引擎不再关心具体的渗透技术细节，只需聚焦于维护当前状态标量，并依据转移函数 δ 进行查表路由。这种设计将不可预测的外部网络对抗环境，规约到了有限的数学空间内，从而赋予了整个自动化调度系统严

格意义上的确定性与极高的鲁棒性。

2.4 软件工程架构范式：微内核与依赖倒置

构建一个支持复杂长链路推演且高容错的 BAS 系统，不仅需要扎实的数学理论，更需要先进的软件架构工程规范。本文的系统设计深度融合了“微内核架构 (Microkernel Architecture)”与“依赖倒置原则 (Dependency Inversion Principle, DIP)”。

2.4.1 微内核架构 (插件化解耦)

微内核架构广泛应用于现代高扩展性软件系统 (如操作系统内核) 的设计中^{richards2020fundamentals}。该架构将整个系统垂直划分为两部分：核心系统 (Core System)：提供系统运行的最小可行性生命周期管理与状态路由。在本文中对对应状态机引擎。插件模块 (Plug-in Modules)：包含特定的业务处理逻辑，可独立扩展而不影响核心系统。在本文中对对应各项具体的攻击能力算子 (Tasks)。具体的渗透动作 (如调用 Hashcat 爆破) 被降级为热插拔的无状态纯函数。这种设计严格遵循了软件工程的“开闭原则 (Open/Closed Principle)”，即系统对功能扩展开放，对核心调度代码封闭。

2.4.2 依赖倒置原则 (IoC 与 DIP)

传统的自动化脚本在编写时，高层业务逻辑模块往往直接调用底层操作系统的物理接口。这种直接依赖导致高层模块极易受到底层网络闪断或磁盘异常的致命影响。依赖倒置原则 (DIP) 的核心思想是：高层模块不应该依赖底层模块，两者都应该依赖其抽象接口^{martin2017clean}。本文通过构建独立的运行时隔离层，实现了彻底的控制反转 (Inversion of Control, IoC)。高层的渗透算子丧失了对底层物理资源的直接操作权限。所有的重试算法、超时熔断与物理层异常捕获均由运行时层收口处理。这种极致的 I/O 隔离设计，阻断了底层异常向核心引擎的逆向传导，极大地提升了系统的存活能力。

第三章 简单微内核状态机架构设计

在明确了入侵与攻击模拟 (BAS) 的演进需求与有限状态机 (FSM) 的理论基础后, 本章将详细论述微内核自动化渗透调度引擎的总体设计思想与架构切分方案。针对长链路渗透测试中“状态易污染、底层强耦合”的工程痛点, 本文提出了一套包含运行时物理隔离、算子无状态化、以及矩阵式状态路由的四层解耦模型。

3.1 总体架构设计思想

传统的自动化渗透测试工具大多采用命令式 (Imperative) 的线性脚本编程, 高度依赖 if-else 或嵌套 while 循环进行控制流的推进。2024 年发表于《Computers Security》的自动化攻击链重构研究表明, 当渗透测试的深度超过三个阶段 (如: 发现 → 提权 → 窃密 → 横向) 时, 命令式脚本的空间复杂度与状态管理难度将呈指数级上升, 极易引发死锁与内存状态崩溃^{li2024statedriven}。为解决此问题, 本系统的总体架构设计确立了以下四大核心指导原则:

(1) 绝对单线程与步进式推演 (Step-by-step Orchestration): 彻底放弃传统并发扫描常用的多线程/协程池模型。在微内核调度域内, 采用单线程轮询机制。引擎在每一滴答 (Tick) 周期内, 仅从持久化数据库中取出一个 Agent 并向前推演单一状态。该设计从根本上消除了共享内存竞争与死锁风险。

(2) 状态驱动模型 (State-Driven Model): 系统的下一步行为不再由硬编码的分支语句决定, 而是由当前主机的“状态标量”决定。引擎只需查表 (状态转移矩阵) 即可完成路由分发。

(3) 彻底的依赖倒置与 I/O 隔离 (I/O Isolation): 将高层攻击逻辑与底层物理操作 (如 HTTP 通信、文件读写、进程拉起) 强行剥离。业务算子仅下发语义指令, 屏蔽底层异常, 从而大幅提高系统的可测试性 (Testability)。

(4) 插件式无副作用算子 (Plug-in Pure Functions): 各个攻击阶段的执行体被严格限制为不包含任何控制流转移权限的“纯函数”。算子仅负责计算并向引擎抛出执行结果事件。

3.2 系统层次模型划分

基于微内核架构 (Microkernel Architecture) 思想^{bass2021software}, 本系统将原本臃肿的渗透控制端垂直切分为职责绝对单一的四层架构模型。自上而下分别为: 总

控调度层、状态路由引擎层、业务算子层与运行时物理层。

(1) 总控调度层 (Main Control Layer) 该层是系统运行的唯一入口，其核心代码为 `main.py`。总控调度器充当了系统“心脏”的角色，内部仅包含一个无限循环的单线程主时钟。其唯一职责是：加载持久化状态数据库 → 拉取全网在线 Agent → 将有效 Agent 送入底层引擎推演 → 统一将状态变迁落盘持久化。

(2) 状态路由引擎层 (FSM Engine Layer) 该层即为系统的微内核 (Core System)，由 `engine.py` 实现。引擎层完全不包含任何渗透测试的具体指令或网络协议解析逻辑。它的工作机制类似于网络路由器，接收总控层传来的 Agent 实体与当前状态，通过查阅“状态转移矩阵”，动态调用对应的业务算子，并根据算子返回的事件 (Event) 计算次态 (Next State)。

(3) 业务算子层 (Tasks Layer) 该层由 `tasks.py` 实现，是整个杀伤链中各战术技术点 (TTPs) 的具体抽象集合。包括环境探测 (Discovery)、提权 (Privilege)、规避 (Evasion)、窃密 (Credential) 与横向移动 (Lateral) 五大算子。每一个算子都是一个独立的插件模块，算子内部不允许包含任何 `sleep` 等阻塞逻辑，也不允许直接建立网络套接字。

(4) 运行时物理层 (Runtime Layer) 作为最底层的防线，运行时层被进一步拆分为本地宿主机运行时 (`io_runtime.py`) 与 API 运行时 (`api_runtime.py`)。

3.3 状态转移矩阵 (Transition Matrix) 设计

状态转移矩阵是整个 FSM 微内核引擎的“灵魂”。在自动机理论中，状态的跃迁可以通过状态转移函数 $\delta(S, E) \rightarrow S'$ 来精确表达，其中 S 为当前状态， E 为触发事件， S' 为次态 [hopcroft2006introduction]。在传统的软件工程实现中，这往往需要编写极其冗长且难以维护的 Switch-Case 语句。本文通过引入数据驱动编程 (Data-driven Programming) 范式，将状态转移函数直接外置为一个二维的 Hash 字典 (即矩阵)。引擎在运行时通过 $\mathcal{O}(1)$ 的时间复杂度进行查表路由。本系统构建的转移矩阵定义了从探测到横向移动的完整攻击链。无论业务算子在执行过程中遭遇何种情况 (如成功、工具缺失、权限不足等)，均被严格坍缩归化为两类语义事件：success 与 error。

该矩阵设计的可以将错误处理策略 (Error Handling Policy) 与业务逻辑解耦。如图 3-3 所示，当 credential 阶段抛出 error 事件时，矩阵不仅指明了路由方向，更起到了一道安全“熔断器 (Circuit Breaker)”的作用，立即将该 Agent 的状态转移

至终止态 `done`，彻底避免了脏数据污染后续的横向移动阶段。

3.4 数据流与控制流流转时序

在微内核无状态化的设计下，业务算子之间如何进行数据传递（例如：窃密算子获取的密码，如何传递给横向移动算子）成为了架构设计的关键。由于传统的全局变量会导致严重的并发污染，本文采用了在上下文中传递元数据（Context Passing）的设计模式 [4]。系统中存在两个核心数据结构贯穿整个生命周期：持久化外壳（State Shell）：包含当前状态 `s` 与元数据包 `ctx`，仅由引擎和总控层操作。上下文元数据（Context Metadata）：即 `ctx` 字典，它是业务算子之间进行跨阶段情报交接的唯一凭证（例如携带 `targetdccrackedpasswordTick`

`main` 层从磁盘 JSON 中反序列化出目标 Agent 的状态外壳。

`main` 层剥离出 `s` 与 `ctx` 交付给 `engine`。

`engine` 查表发现当前处于 `credential`（窃密）阶段，便将目标 Agent 的凭证特征与 `ctx` 通过纯函数参数传递给 `Tasks` 算子层。

窃密算子请求 `Runtime` 层下发内存转储 API，`Runtime` 负责网络阻塞等待并返回底层的 Base64 回显结果。

窃密算子通过正则表达式提取出明文密码，将其写入 `ctx['crackedppassword']success`

`engine` 接收到更新后的 `ctx` 与事件，查阅矩阵判定下一阶段为 `lateral`，修改外壳标量，返回给 `main`。

`main` 再次序列化数据，执行原子化落盘（磁盘 I/O 保存至 `state.json`）。

得益于上述严谨的数据流闭环设计与运行时层的重试机制，当控制台在任意时刻发生中断并重启时，引擎均能完美复现出上一周期的终端态。如下面的运行日志截图所示，系统的状态迁跃完全符合矩阵预期，展示了极高的工程健壮性。

第四章 系统核心模块的工程实现

本章立足于前文提出的微内核调度模型，详细阐述系统的工程落地细节。本章首先探讨底层运行时环境 (Runtime) 的隔离封装策略，随后剖析核心算子 (Tasks) 的事件触发与标准化构造规范。通过这一套高内聚、低耦合的底层框架，系统成功解决了传统渗透脚本中常见的逻辑混杂与状态失控问题，为上层 8 大攻击能力的稳定执行提供了坚实的运行环境。

4.1 底层运行时环境 (Runtime) 的隔离封装机制

Runtime 层是本系统实现“微内核”架构的物理基础。在传统的渗透测试脚本中，攻击逻辑通常直接调用 requests 或 subprocess 等库，导致业务代码与物理环境深度耦合。根据 Robert C. Martin 提出的依赖倒置原则 (Dependency Inversion Principle, DIP) martin2017clean，高层模块 (业务算子) 不应依赖低层模块 (网络/文件系统)，两者都应依赖其抽象。为此，本系统构建了完全隔离的 `apiruntimeiorruntimeSide-effects`

4.1.1 API 运行时的目标隔离与重试机制 (`apirruntime.py`)

API 运行时是系统与远程 Caldera C2 服务器通信的唯一窗口。针对分布式环境中不稳定的网络状态与复杂的任务调度逻辑，本文实现了以下核心机制：(1) 任务下发的“专属组隔离”策略在 Caldera 的原生机制中，任务 (Operation) 是面向“组 (Group)”下发的。若多个 Agent 处于同一组内，下发一次横向移动指令会导致组内所有主机同步发起攻击，造成流量特征剧烈波动甚至系统逻辑死锁 (即前文提到的“核裂变感染”)。工程实现：在 `launchopPATCH/api/v2/agents/pawAgentgroupAgentpawBulk`

4.1.2 I/O 运行时的副作用收口与安全控制 (`iorruntime.py`)

`iorruntime1.runcmdHashcatPypykatzsubprocess.runtimeout(stdout, stderr, returncode);`
`datasass.dmpos.path.basename.././../etc/passwdasp2024path`

4.2 核心算子的事件触发与标准化构造 (Tasks)

算子层 (Tasks) 是系统架构中变动最频繁、业务逻辑最复杂的层次。为了确保在增加新的攻击能力时不影响引擎稳定性，本系统参照了函数式编程中的“纯函数”概念与事件驱动架构 (Event-Driven Architecture) richards2020fundamentals。

4.2.1 算子的标准化接口设计模型

所有的攻击步骤 (Discovery, Privilege, Evasion...) 都被定义为遵循统一函数签名的“无状态算子”：`def task_function(agent, ctx) -> (event, new_ctx)` *Input agent : IP, Paw, OSctx(Context) : DCIPOutput event : success/error new_ctx : IoC*

4.2.2 状态推演的事件映射逻辑 (Event Normalization)

算子的核心工作是将混乱的、非结构化的工具回显 (Unstructured Data) 转化为结构化的状态机事件。(1) 成功路径的归约：例如在 `credential` (凭证窃取) 算子中，工具返回的可能是数百行的 LSASS 内存解析日志。算子内部利用正则表达式进行模式匹配：`re.search(r'(?:(NT|NTLM):*([a-zA-F0-9]{32})', output)` 一旦匹配到哈希，算子便将该值注入 `ctx`，并触发 `success` 事件。这一过程将复杂的黑客工具逻辑封装在了算子内部，对外部引擎呈现的是干净的布尔逻辑。(2) 失败路径的防御性设计：如果正则表达式匹配失败，或者 API 返回结果为空，算子会主动抛出 `error` 事件。(3) 容灾逻辑：在 `lateral` (横向移动) 算子中，系统会依次尝试 WinRM 和 WMI。这种“备选方案”逻辑被封装在算子内部，只有当所有尝试都失败时，才向引擎汇报 `error`。

4.2.3 上下文 (Context) 的单向流动与持久化协议

为了支持“断点续传”，上下文 `ctx` 的构造采用了扁平化设计。状态快照：每次算子返回 `new_ctx.JSONdb.save`

4.3 渗透杀伤链 8 个核心能力 (Abilities) 底层原理剖析

本系统在微内核引擎的调度下，依序执行了映射自 MITRE ATTCK 框架的 8 个战术技术能力 (Abilities)。在自动化的表象之下，这些算子实际上是在与 Windows 操作系统的核心机制 (如进程完整性、身份认证子系统、驱动过滤机制) 进行深度的博弈。以下将结合具体的注入脚本，对其底层机制进行极尽详细的剖析。

4.3.1 阶段一：域环境侦察与微隔离定位 (Discovery)

Ability 1: host_discovery(SRV - T1018)

代码执行逻辑：该能力放弃了 Nmap 等传统的外部端口扫描工具，完全依赖 Windows 原生的 PowerShell cmdlet。执行流首先利用 `Get-CimInstance Win32_ComputerSystemMemberWorkgroup` 获取域控信息，然后利用 `Get-NetIPAddress -InterfaceIndex 1 -AddressFamily IPv4` 获取 IP 地址，最后通过 `Get-NetRoute -InterfaceIndex 1 -DestinationPrefix 0.0.0.0/0` 获取路由表信息。通过这些信息，可以确定域控的 FQDN (完全限定域名)。

操作系统与网络协议底层原理：在 Windows Active Directory (AD) 架构中，域控

(DC) 不仅是身份验证中心, 也是动态 DNS 服务器。当一台新的 Windows 主机加入域, 或者用户发起 Kerberos 登录请求时, 系统底层 (Netlogon 服务) 必须知道应该把票据请求发给谁。微软的解决方案是: 要求域控在启动时, 强制向 DNS 注册一系列的服务定位器记录 (SRV Records)。*msdcsldap.tcp.dcLDAP[1]DNSUDP53NTALivingofftheL*

- 4.3.2 标题内容
- 4.3.3 标题内容
- 4.3.4 标题内容
- 4.3.5 标题内容

致 谢

参考文献